

Incident Reporting System

Project Report

A database-driven platform to report, track and resolve incidents

NAME	Usman Aslam & Abdullah Amjad
ID	(41566 & 41195)
COURSE	Database Systems
Submitted to	Ma'am Tayyaba Shehzad
Technology	MySQL · Python (Flask) · HTML / CSS / JavaScript

Table of Contents

1. Abstract	3
2. Introduction	3
3. Problem Statement	3
4. Objectives.....	3
5. System Requirements	4
5.1 Software Requirements	4
5.2 Hardware Requirements.....	4
6. Functional Requirements.....	4
7. System Architecture.....	4
8. Database Design.....	5
8.1 Entity-Relationship Diagram	6
8.2 Database Tables	6
8.3 Relationships	8
8.4 Normalization.....	9
8.5 Constraints and Referential Integrity	9
9. Data Flow Diagram.....	9
10. Incident Lifecycle	10
11. Implementation	11
11.1 Technology Stack	11
11.2 Key SQL Queries	11
12. Results (Screenshots).....	11
13. Conclusion	14

1. Abstract

The Incident Reporting System is a database-powered platform designed to manage and organize incidents within an organization. It provides a structured approach for reporting incidents, tracking their status, and resolving issues in an efficient and organized way. The system is built on a relational MySQL database and allows users to report incidents, administrators to review and manage them, and support staff to update their status and record resolutions.

By storing every incident, verification, resolution and notification in a set of related database tables, the system helps an organization handle incidents quickly, maintain proper records, and improve overall response time, transparency and accountability.

2. Introduction

In many organizations, different types of incidents can occur, such as technical problems, security issues, or workplace accidents. Managing these incidents manually is difficult because information may be lost, records may not be organized, and resolving problems can take a long time.

Often incidents are reported through phone calls, emails, or paper forms, which makes it harder to track their status and progress. This creates confusion and delays in solving the issues. The Incident Reporting System solves this problem by providing a digital platform where users can easily report incidents, administrators can review these reports and assign them to the responsible staff, and the reporter is kept informed at every step.

3. Problem Statement

In many organizations, incidents are reported and managed manually through paper records, emails, or verbal communication. This method causes delays, missing information, and difficulty in tracking the status of incidents. Because there is no proper system to record and monitor incidents, it becomes hard for administrators to manage them efficiently and to ensure that problems are resolved on time.

The Incident Reporting System solves this problem by providing a centralized database where incidents can be reported, stored, tracked, and resolved in an organized way, improving efficiency, transparency and record management.

4. Objectives

- **Provide a secure platform** for users and administrators to log in.

- **Allow users to report incidents** with full details (category, severity, location).
- **Enable tracking** of each incident's status from report to resolution.
- **Let administrators manage incidents** by verifying, assigning and resolving them.
- **Keep users informed** through automatic notifications on every status change.
- **Store all records securely** in a well-designed relational database.

5. System Requirements

5.1 Software Requirements

- **Database:** MySQL 8.0 (relational database engine).
- **Backend:** Python with the Flask web framework (using raw SQL).
- **Frontend:** HTML, CSS and JavaScript.
- **Tools:** Visual Studio Code and a MySQL client.

5.2 Hardware Requirements

- **Processor:** any modern dual-core processor or better.
- **Memory:** minimum 4 GB RAM.
- **Storage:** around 2 GB free disk space for the database engine and data.

6. Functional Requirements

- **User Authentication** — secure login and registration for users and administrators.
- **Incident Reporting** — users can report incidents by submitting details through the system.
- **Incident Tracking** — users can track the current status of their reported incidents.
- **Incident Management** — administrators can review, verify, assign and manage incidents.
- **Status Updates** — staff can update incident progress and record resolution details.
- **Notifications** — the system notifies users automatically when their incident changes.
- **Database Management** — secure storage of incident records and user information.

7. System Architecture

The system follows a classic three-tier architecture. The presentation layer (the web pages in the browser) sends requests to the application layer (the Flask server), which runs SQL queries against the data layer (the MySQL database) and returns the results to the user.

System Architecture (3-Tier)

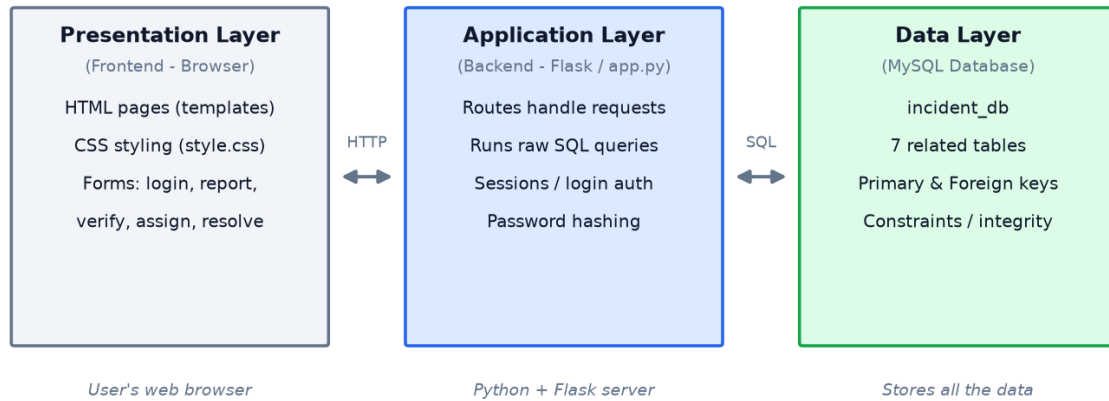


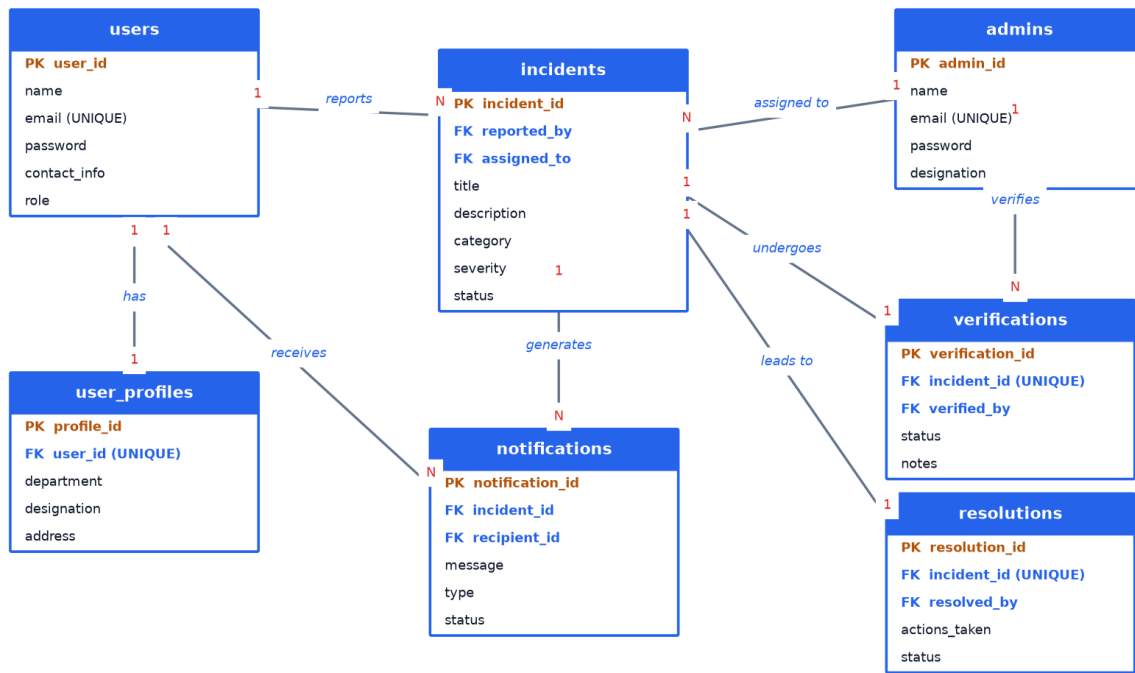
Figure 1: Three-tier system architecture.

8. Database Design

The database is the core of this project. It is named `incident_db` and contains seven related tables. The design uses primary keys to identify rows uniquely and foreign keys to link the tables together, enforcing referential integrity.

8.1 Entity-Relationship Diagram

Entity-Relationship Diagram



PK = Primary Key FK = Foreign Key 1:1 / 1:N = relationship cardinality

Figure 2: Entity-Relationship (ER) diagram of the database.

8.2 Database Tables

The following tables describe each entity, its columns, and the keys and constraints used.

Table 1 — users (people who report incidents)

Column	Type	Key / Constraint	Description
user_id	INT	PRIMARY KEY	Unique id of the user (auto-increment).
name	VARCHAR(100)	NOT NULL	Full name of the user.
email	VARCHAR(120)	UNIQUE, NOT NULL	Login email; no two users share one.
password	VARCHAR(255)	NOT NULL	Password stored in hashed form.
contact_info	VARCHAR(50)	—	Phone or contact number.
role	VARCHAR(20)	DEFAULT 'Reporter'	Role of the user.
created_at	TIMESTAMP	DEFAULT now	When the account was created.

Table 2 — user_profiles (extra details, 1:1 with users)

Column	Type	Key / Constraint	Description
profile_id	INT	PRIMARY KEY	Unique id of the profile.
user_id	INT	FK → users, UNIQUE	Links to one user (UNIQUE makes it 1:1).
department	VARCHAR(80)	—	Department of the user.
designation	VARCHAR(80)	—	Job title of the user.
address	VARCHAR(200)	—	Address of the user.

Table 3 — admins (administrators / support staff)

Column	Type	Key / Constraint	Description
admin_id	INT	PRIMARY KEY	Unique id of the administrator.
name	VARCHAR(100)	NOT NULL	Name of the administrator.
email	VARCHAR(120)	UNIQUE, NOT NULL	Login email of the administrator.
password	VARCHAR(255)	NOT NULL	Hashed password.
designation	VARCHAR(50)	DEFAULT 'Administrator'	Admin or Support Staff.

Table 4 — incidents (the central table)

Column	Type	Key / Constraint	Description
incident_id	INT	PRIMARY KEY	Unique id of the incident.
reported_by	INT	FK → users	The user who reported the incident.
title	VARCHAR(150)	NOT NULL	Short title.
description	TEXT	NOT NULL	Full description of the incident.
category	ENUM	NOT NULL	Technical / Security / Workplace / Other.
severity	ENUM	NOT NULL	Low / Medium / High / Critical.
location	VARCHAR(120)	—	Where the incident happened.
status	ENUM	DEFAULT 'Pending'	Current stage in the lifecycle.
assigned_to	INT	FK → admins, NULL	Admin handling it (NULL until assigned).
reported_at	TIMESTAMP	DEFAULT now	When it was reported.
assigned_at	TIMESTAMP	NULL	When it was assigned.

Table 5 — verifications (1:1 with incidents)

Column	Type	Key / Constraint	Description
verification_id	INT	PRIMARY KEY	Unique id of the verification.
incident_id	INT	FK → incidents, UNIQUE	The incident verified (UNIQUE = 1:1).
verified_by	INT	FK → admins	Admin who verified it.
status	ENUM	NOT NULL	Verified / Rejected.
notes	TEXT	—	Optional notes.
verified_at	TIMESTAMP	DEFAULT now	When it was verified.

Table 6 — resolutions (1:1 with incidents)

Column	Type	Key / Constraint	Description
resolution_id	INT	PRIMARY KEY	Unique id of the resolution.
incident_id	INT	FK → incidents, UNIQUE	The incident resolved (UNIQUE = 1:1).
resolved_by	INT	FK → admins	Responsible admin.
actions_taken	TEXT	NOT NULL	What was done to resolve it.
status	ENUM	DEFAULT 'Resolved'	Resolved / Closed / Reopened.
completion_date	TIMESTAMP	DEFAULT now	When it was resolved.

Table 7 — notifications (1:N with incidents and users)

Column	Type	Key / Constraint	Description
notification_id	INT	PRIMARY KEY	Unique id of the notification.
incident_id	INT	FK → incidents	The incident it is about.
recipient_id	INT	FK → users	The user who receives it.
message	VARCHAR(255)	NOT NULL	The notification text.
type	ENUM	DEFAULT 'System'	System / Email / SMS.
status	ENUM	DEFAULT 'Unread'	Unread / Read.
created_at	TIMESTAMP	DEFAULT now	When it was created.

8.3 Relationships

The tables are connected through the following relationships:

- **users — user_profiles (1:1)**: each user has at most one profile (UNIQUE foreign key).
- **users — incidents (1:N)**: one user can report many incidents.

- **admins — incidents (1:N):** one admin can be assigned many incidents.
- **incidents — verifications (1:1):** each incident has one verification.
- **incidents — resolutions (1:1):** each incident has one resolution.
- **incidents — notifications (1:N):** one incident can generate many notifications.
- **users — notifications (1:N):** one user can receive many notifications.

8.4 Normalization

The database is designed in Third Normal Form (3NF):

- **First Normal Form (1NF):** every column holds a single, atomic value — no repeating groups.
- **Second Normal Form (2NF):** each table has a single-column primary key, so every non-key column depends on the whole key.
- **Third Normal Form (3NF):** non-key columns depend only on the primary key. For example, an admin's name is not copied into the incidents table; only the admin's id is stored and the name is looked up with a JOIN.

8.5 Constraints and Referential Integrity

- **PRIMARY KEY:** uniquely identifies each row.
- **FOREIGN KEY:** links a row to a row in another table and enforces referential integrity.
- **UNIQUE:** prevents duplicate values (e.g. email) and enforces 1:1 relationships.
- **NOT NULL / DEFAULT:** ensure required columns always have a valid value.
- **ENUM:** restricts a column to a fixed list of valid values (e.g. severity).
- **ON DELETE CASCADE / SET NULL:** keep the data consistent when a parent row is deleted.

9. Data Flow Diagram

The data flow diagram shows how information moves through the system: a user reports an incident (stored in the incidents table); an admin verifies it (verifications table), then assigns and resolves it (resolutions table); and notifications are generated for the user.

Data Flow Diagram

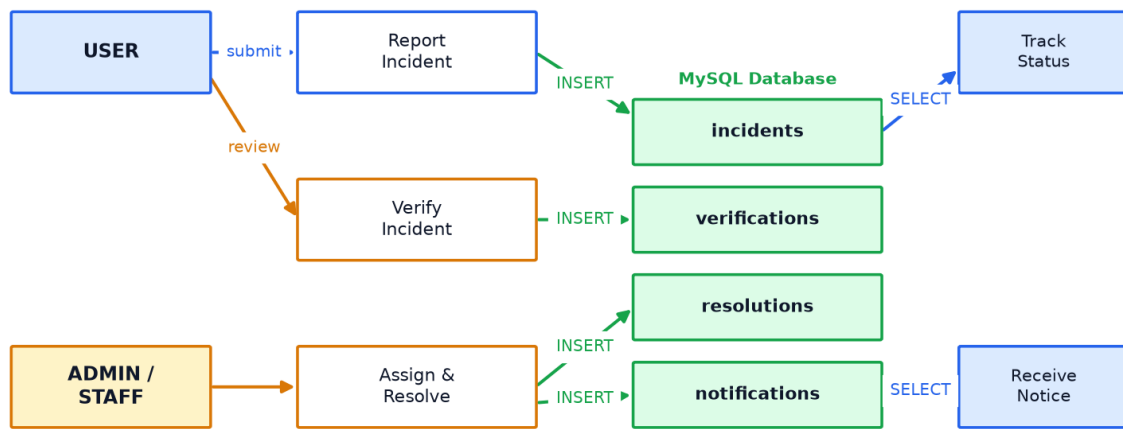


Figure 3: Data flow diagram.

10. Incident Lifecycle

Every incident moves through a clear set of statuses. A newly reported incident is Pending. After an admin reviews it, it becomes Verified (or Rejected). It is then Assigned to a staff member and finally marked Resolved. Each change updates the status column and notifies the reporter.

Incident Status Lifecycle

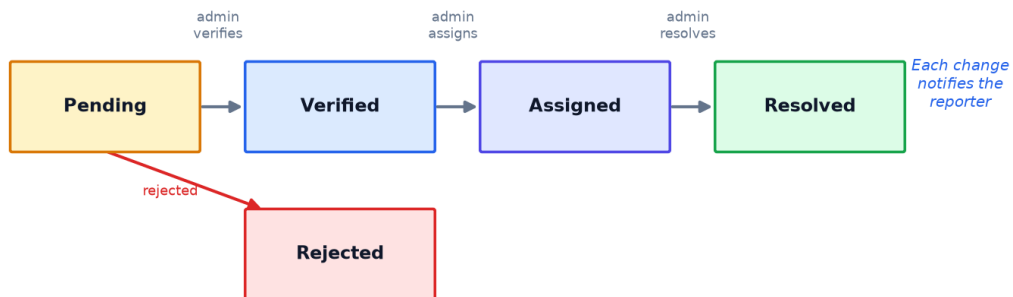


Figure 4: Incident status lifecycle.

11. Implementation

11.1 Technology Stack

The application is built with Python and the Flask web framework. Flask was chosen over an ORM-based framework so that every database operation is written as raw SQL that is fully visible in the code. The database is MySQL, and the user interface uses HTML, CSS and JavaScript.

11.2 Key SQL Queries

Reporting a new incident (INSERT):

```
INSERT INTO incidents (reported_by, title, description, category, severity, location) VALUES (%s, %s, %s, %s, %s, %s);
```

Showing a user only their own incidents (SELECT with WHERE):

```
SELECT incident_id, title, status FROM incidents WHERE reported_by = %s ORDER BY reported_at DESC;
```

Joining an incident with its reporter and assigned admin (JOIN):

```
SELECT i.*, u.name AS reporter_name, a.name AS admin_name FROM incidents i JOIN users u ON i.reported_by = u.user_id LEFT JOIN admins a ON i.assigned_to = a.admin_id WHERE i.incident_id = %s;
```

Counting incidents by status (GROUP BY):

```
SELECT status, COUNT(*) AS c FROM incidents GROUP BY status;
```

12. Results (Screenshots)

The following screenshots show the working system.

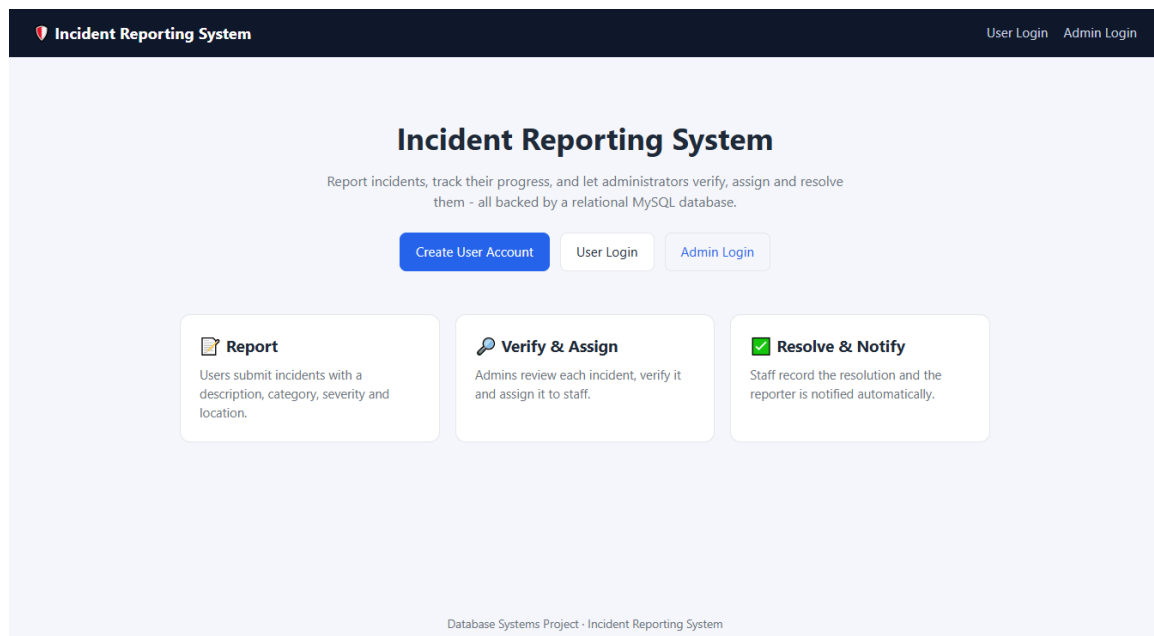


Figure 5: Home page.

The screenshot shows the home page of the Incident Reporting System. At the top is a dark blue navigation bar with the system name on the left and links for Dashboard, Report Incident, Notifications, and the user's name Alice Johnson on the right, along with a Logout button. The main content area has a light blue background. It features a section titled 'My Reported Incidents' with a '+ Report New Incident' button. Below this is a table listing reported incidents. The table has columns for ID, Title, Category, Severity, Status, and Reported. Two incidents are listed: one with ID #3 titled 'Test projector broken' (Technical, High severity, Resolved status) and one with ID #1 titled 'Wi-Fi not working in Block A' (Technical, Medium severity, Pending status). Each row has a 'View' link. At the bottom of the page is a footer that reads 'Database Systems Project - Incident Reporting System'.

ID	TITLE	CATEGORY	SEVERITY	STATUS	REPORTED
#3	Test projector broken	Technical	High	Resolved	13 Jun 2026
#1	Wi-Fi not working in Block A	Technical	Medium	Pending	13 Jun 2026

Figure 6: User dashboard listing the user's reported incidents.

The screenshot shows the 'Report a New Incident' form. It is a white card on a light blue background. The form has a title 'Report a New Incident' and several input fields: 'Title' (with placeholder 'Short summary'), 'Description' (with placeholder 'Describe what happened...'), 'Category' (a dropdown menu currently showing 'Technical'), 'Severity' (a dropdown menu currently showing 'Low'), and 'Location' (with placeholder 'e.g. Block A, Server Room'). At the bottom of the form is a blue 'Submit Incident' button. The same dark blue navigation bar and footer as in Figure 6 are present.

Figure 7: Reporting a new incident.

Incident Reporting System

[Dashboard](#)[Report Incident](#)[Notifications](#)[Alice Johnson](#)

Logout

[← Back to dashboard](#)

#3 · Test projector broken

Resolved

Projector in Lab 3 won't turn on.

CATEGORY
Technical

SEVERITY
High

LOCATION
Lab 3

REPORTED BY
Alice Johnson

REPORTED AT
13 Jun 2026, 23:56

HANDLED BY
Sara Khan

Progress

1. Verification

Marked **Verified** by System Admin on 13 Jun 2026.
Note: Checked with lab staff.

2. Resolution

Resolved by System Admin on 13 Jun 2026.
Actions taken: Replaced the projector power cable.

Database Systems Project · Incident Reporting System

Figure 8: Incident tracking with verification and resolution.

Incident Reporting System

[Dashboard](#)[Report Incident](#)[Notifications](#)[Alice Johnson](#)

Logout

Notifications

Your incident has been assigned to a staff member.
Incident: Test projector broken · 13 Jun 2026, 23:56 · System

Good news - your incident has been Resolved.
Incident: Test projector broken · 13 Jun 2026, 23:56 · System

Your incident was reviewed and marked: Verified.
Incident: Test projector broken · 13 Jun 2026, 23:56 · System

Your incident was submitted and is now Pending review.
Incident: Test projector broken · 13 Jun 2026, 23:56 · System

Your incident was submitted and is now Pending review.
Incident: Wi-Fi not working in Block A · 13 Jun 2026, 23:55 · System

Database Systems Project · Incident Reporting System

Figure 9: User notifications.

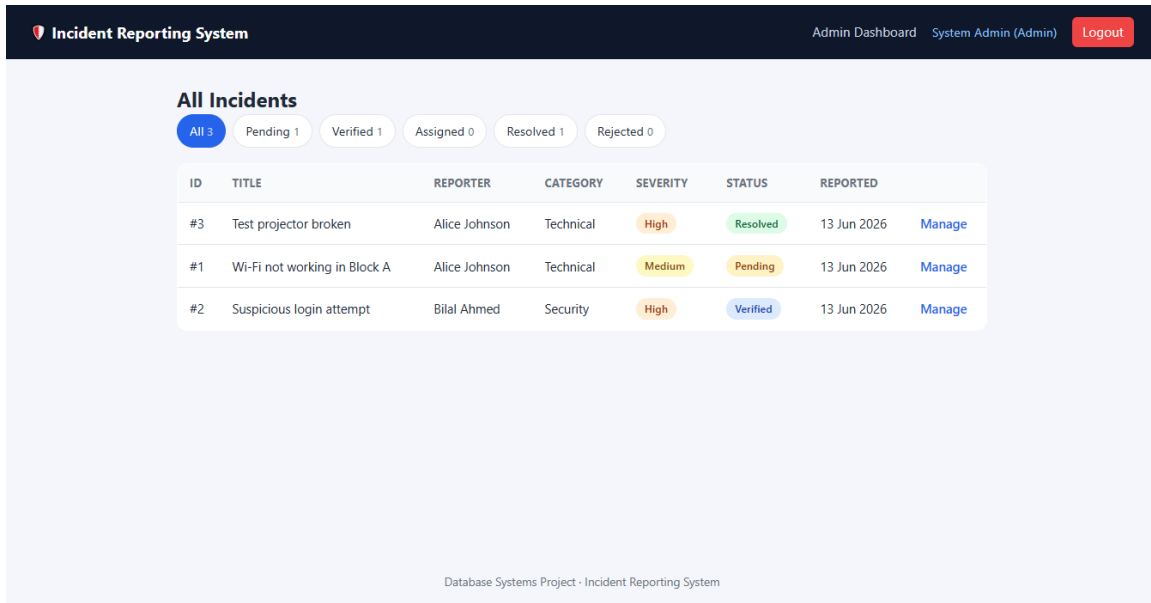


Figure 10: Admin dashboard with status filters and counts.

Incident Reporting System Admin Dashboard System Admin (Admin) Logout

[← Back to all incidents](#)

#1 - Wi-Fi not working in Block A Pending

The wireless network keeps disconnecting every few minutes.

CATEGORY	SEVERITY	LOCATION	REPORTER
Technical	Medium	Block A	Alice Johnson (alice@example.com)

REPORTED AT	ASSIGNED TO
13 Jun 2026, 23:55	Nobody yet

1. Verify

Decision

Verified (valid incident)

Notes

Optional notes

Save Verification

2. Assign

Assign to staff

Sara Khan

Assign

3. Resolve

Actions taken

Describe how it was resolved

Mark Resolved

Database Systems Project - Incident Reporting System

Figure 11: Admin managing an incident (verify, assign, resolve).

13. Conclusion

The Incident Reporting System provides an organized and efficient way to record and handle incidents within an organization. By using a relational MySQL database of seven well-designed tables, the system ensures that incident information is stored securely and can be accessed easily when required. Primary and foreign keys enforce data integrity, the design is normalized

to Third Normal Form, and constraints keep the data valid. Overall, the system improves response time, transparency and the management of incidents, making it a useful tool for any organization that wants to manage incidents in a structured and reliable way.